CP 312 Assignment 2

I certify that this Assignment is entirely my work and that all external resources, if used, are properly acknowledged

1.

$T(n) = 2T(\frac{n}{10}) + \sqrt{n}$ (Base Case: $T(n) = 17$ for $n < 1$)

Recurrance Tree method:

Assume n is a power of 10

↳ Level 0: cost is $\sqrt{n}$

↳ Level 1: 2 subproblems of size $\frac{n}{10}$. Cost $= 2 \times \sqrt{\frac{n}{10}} = \frac{2}{\sqrt{10}}\sqrt{n}$

↳ Level 2: 4 subproblems of size $\frac{n}{100}$. Cost $= 4 \times \sqrt{\frac{n}{100}} = (\frac{2}{\sqrt{10}})^2 \sqrt{n}$

↳ Level i: the cost is $(\frac{2}{\sqrt{10}})^i \sqrt{n}$

Total cost is sum of all levels:

$$T(n) = \sum_{i=0}^{\log_{10} n} (\frac{2}{\sqrt{10}})^i \sqrt{n} = \sqrt{n} \sum_{i=0}^{\log_{10} n} (\frac{2}{\sqrt{10}})^i$$

This is a geometric series with ratio $r = \frac{2}{\sqrt{10}} \approx \frac{2}{3.16} = 0.63$.

Since $r < 1$, the series is converging to a constant:

$$\sum_{i=0}^{\infty} r^i = \frac{1}{1-r}$$

∴ the total cost is dominated by the first term: $T(n) = \Theta(\sqrt{n})$

Master Theorem verification:

Recurrace form: $T(n) = aT(n/b) + f(n)$

Critical exponent: $\log_b a = \log_{10} 2 \approx 0.301$

$f(n) = n^{0.5}$

$n^{\log_b a} = n^{0.301}$

Since $0.5 > 0.301$, $f(n)$ grows polynomially faster than $n^{\log_b a}$.

This corresponds to case 3 of the Master Theorem.

↳ Must satisfy $af(\frac{n}{b}) \leq cf(n)$ for some constant $c < 1$ and sufficiently large n.

$$2\sqrt{\frac{n}{10}} \leq c\sqrt{n}$$

$$\frac{2}{\sqrt{10}}\sqrt{n} \leq c\sqrt{n}$$

$$\frac{2}{\sqrt{10}} \approx 0.632 \leq c$$ (This holds for any c where $0.632 \leq c < 1$)

∴ By the Master theorem (case 3), $T(n) = \Theta(f(n)) = \Theta(\sqrt{n})$

2.

Recurrance: $T(n) = \sqrt{n}\, T(\frac{n}{2}) + 1$

Let $n = 2^m$. Then $m = \log_2 n$. Sub into Recurrance:

$$T(2^m) = \sqrt{2^m}\, T(\frac{2^m}{2}) + 1$$

$$T(2^m) = 2^{m/2}\, T(2^{m-1}) + 1$$

Let $S(m) = T(2^m)$. Recurrance becomes:

$$S(m) = 2^{m/2}\, S(m-1) + 1$$

$$S(m) = 2^{m/2}\left[2^{(m-1)/2}\, S(m-2) + 1\right] + 1$$

$$S(m) = 2^{m/2 + (m-1)/2}\, S(m-2) + 2^{m/2} + 1$$

$$S(m) = 2^{\frac{m+(m-1)}{2}}\, S(m-2) + 2^{m/2} + 1$$

continuing k times:

$$\frac{1}{2}\sum_{j=1}^{m} j = \frac{1}{2}\,\frac{m(m+1)}{2} = \frac{m^2+m}{4}$$

$$S(m) = 2^{\frac{m^2}{4}}$$

$$T(n) = 2^{\frac{(\log n)^2}{4}} = (2^{\log n})^{\frac{\log n}{4}} = n^{\frac{\log n}{4}}$$

Substitution Method:

$$T(n) \leq n^{1/2}\, T(\frac{n}{2}) + 1$$

$$T(n) \leq n^{1/2} \cdot c(\frac{n}{2})^{\frac{\log n - 1}{4}} + 1$$

$$T(n) \leq c \cdot n^{1/2} \cdot \frac{n^{\frac{\log n}{4}}}{2^{\frac{\log n}{4}}} + 1$$

∴ This confirms that the super polynomial growth rate $O(n^{\frac{\log n}{4}})$

3.

```
Function Find Valley (A, n):
    low = 1
    high = n
    while low < high:
        mid = floor((low + high) / 2)
        if A[mid] < A[mid + 1]:
            high = mid
        else:
            low = mid + 1
    return low
```

Algorithim Strategy:

I used a modified binary search to find the local minimum (the "valley"). Since the array decreases and then increases, checking the slope at the middle element tells me which direction the minimum lies.

↳ If A[mid] < A[mid + 1], we are in the ascending part (or valley). The minimum is to the left (or is mid)

↳ If A[mid] > A[mid + 1], we are in the descending part. The minimum is to the right

- The comparision A[mid] < A[mid + 1] takes O(1) time
- The recurrance for the running time is $T(n) = T(n/2) + c$
- By Master Theorem case 2, this resolves to $O(\log n)$
- Since $\log n \in o(n)$, this satisfies the strict sublinear requirement

4.

```
Function CountWeightedInversions(A):
    n = length(A)
    if n ≤ 1
        return 0
    mid = n/2
    Left = A[0...mid-1]
    Right = A[mid...n-1]
    count = CountWeightedInversions(Left) + CountWeightedInversions(Right)
    Sort(Left)
    Sort(Right)
    j = 0
    for i from 0 to length(Left) - 1:
        While j < length(Right) and Left[i] > 2 * Right[j]:
            j = j+1
        count = count + j
    Merge(A, Left, Right)
    return count
```

Algorithim Strategy:

Used the divide and conquer approach similar to Merge Sort

↳ Divide array into 2 halves

↳ Recursively count inversions in the left and right halves

↳ Merge step: count split inversions (where i is in the left, j is in the right) and then merge the sorted halves.

- The SplitCount loop runs in $O(n)$ because i and j only increment and never backtrack
- The Merge step takes $O(n)$
- Recurrance: $T(n) = 2T(\frac{n}{2}) + O(n)$
- By Master Theorem Case 2, $T(n) = O(n \log n)$

5.

```
Function 4Sum(A, Target):
    Sort(A)
    Solutions = []
for i from 0 to n-4:
    if i>0 and A[i] == A[i-1]: continue
    NewTarget = Target - A[i]
    for j from i+1 to n-3:
        left = j+1
        right = n-1
        While left < right:
            Current_Sum = A[i] + A[left] + A[right]
            if Current_Sum == NewTarget:
                Solutions.add({A[i], A[j], A[left], A[right]})
                left ++, right --
            else if Current_Sum < NewTarget:
                left ++
            else:
                right --
    return Solutions
```

Algorithm:

given 3sum which runs in $O(n^2)$. Reduce 4sum to n instances of 3Sum

1. Sort the Array A. Time: $O(n \log n)$
2. Iterate through the array with index i from 0 to n-4
3. Fix the first elemcat as A[i]
4. The problem now becomes finding 3 elemets in the Subarray A[i+1...n-1] that sums upto Target - A[i]
5. Call 3Sum subroutine on the subarray with the new target

- Sorting: $O(n \log n)$. outer loop runs n times
- Inside loop preforms a 3sum operation on a subarray size $\approx n$
- 3Sum takes $O(n^2)$ time
- Total complexitiy $\approx O(n \log n) + n \times O(n^2) = O(n^3)$